

RTL Engineering Shadow

Product Discovery and Design Documentation

Sayyam Palrecha

July 15, 2026

Contents

1	Engineering Memory Manifesto	2
1.1	Purpose	2
1.2	The Problem	2
1.3	Our Beliefs	3
1.4	Design Principles	3
1.5	Vision	3
1.6	Success	4
2	Vision	4
3	Problem Statement	4
4	Primary User Persona	4
5	Current Workflow	4
6	Engineering Shadow Behavioral Specification	4
6.1	Purpose	4
6.2	Behavioral Philosophy	5
6.3	Behavioral Principles	5
6.4	Behavioral Summary	7
7	Product Hypothesis	7
8	MVP	7
9	Open Questions	8

1 Engineering Memory Manifesto

1.1 Purpose

This project exists to address a fundamental limitation in modern engineering workflows: the inability to preserve and communicate engineering understanding as projects evolve.

Engineering projects are not defined solely by their source code, documentation, or design artifacts. They are driven by a continuous sequence of decisions, assumptions, experiments, trade-offs, and evolving understanding. Most of this knowledge exists only in the engineer's mind and gradually disappears over time.

The objective of this project is to preserve this engineering understanding in a structured, continuously evolving form.

1.2 The Problem

Modern engineering tools successfully manage artifacts.

Examples include:

- Source code
- Design files
- Version history
- Documentation
- Simulation results
- Reports

However, these tools rarely preserve the reasoning that produced those artifacts.

Questions such as the following often remain unanswered:

- Why was a particular architecture selected?
- Which alternatives were considered?
- What constraints influenced the decision?
- Why was an existing solution rejected?
- How did the design evolve over time?

As projects grow, this missing context becomes increasingly difficult to reconstruct.

1.3 Our Beliefs

We believe that:

1. Engineering knowledge extends beyond source code.
2. Design decisions are first-class engineering artifacts.
3. Engineering context continuously evolves throughout the project lifecycle.
4. Preserving reasoning is as important as preserving implementation.
5. Documentation should describe understanding rather than merely recording activities.
6. Engineering knowledge should remain accessible without relying on human memory.
7. Knowledge captured once should be reusable throughout future projects.

1.4 Design Principles

Every future design decision within this project should follow the principles below.

1. Minimize interruption to the engineer's workflow.
2. Preserve context rather than simply recording events.
3. Prefer understanding over observation.
4. Treat engineering decisions as persistent project knowledge.
5. Allow engineering knowledge to evolve as designs evolve.
6. Generate documentation from understanding rather than generating understanding from documentation.
7. Maintain transparency regarding captured knowledge and generated outputs.
8. Ensure that every generated artifact can be traced back to supporting engineering context.

1.5 Vision

We envision engineering environments where project knowledge no longer disappears as engineers move between tasks, meetings, iterations, or team members.

Instead, engineering understanding continuously evolves alongside the project itself, enabling efficient collaboration, onboarding, design reviews, and long-term knowledge preservation.

1.6 Success

This project will be considered successful if it enables engineers to communicate project context, design evolution, and engineering reasoning with significantly less effort than is required using current engineering workflows.

Success is measured not by the amount of generated documentation, but by the quality, continuity, and accessibility of preserved engineering understanding.

2 Vision

Status: Draft

TODO:

- Long-term vision
- Mission
- North Star

3 Problem Statement

Status: Empty

Describe the engineering problem, why it exists, who experiences it, and why current tools fail.

4 Primary User Persona

- RTL Design Engineer
- PhD Student
- Hardware Design Team

5 Current Workflow

Capture the user's real RTL workflow before proposing solutions.

6 Engineering Shadow Behavioral Specification

6.1 Purpose

This document defines the expected behavior of Engineering Shadow independent of implementation details.

Its purpose is to ensure that every implementation of Engineering Shadow exhibits consistent behavior regardless of the underlying technologies, programming languages, AI models, or deployment environments.

The principles defined in this document represent permanent behavioral characteristics of the product and shall guide all future product, architecture, and engineering decisions.

6.2 Behavioral Philosophy

Engineering Shadow exists to reduce the cognitive overhead associated with long-term RTL engineering projects.

The Shadow does not replace the engineer.

The Shadow continuously develops and maintains an evolving understanding of the project so that engineering assistance becomes increasingly context-aware as the project progresses.

The Shadow's objective is not to maximize automation.

Its objective is to maximize engineering understanding while minimizing interruption.

6.3 Behavioral Principles

Principle 1: Minimize Cognitive Load

Statement

Every interaction initiated by the Shadow shall reduce, or at minimum not increase, the engineer's cognitive load.

Rationale

Engineers already operate under significant cognitive demand while solving complex design problems. Excessive prompts, notifications, or unnecessary interactions reduce productivity and discourage long-term use.

Example

The Shadow remains silent while the engineer edits source code.

The Shadow asks for clarification only after detecting a significant engineering event.

Principle 2: Understanding Before Assistance

Statement

The Shadow shall establish sufficient project understanding before providing project-specific recommendations.

Rationale

Recommendations generated without sufficient context are often generic, inconsistent with project constraints, or technically incorrect.

Project understanding shall always precede project-specific assistance.

Example

Before recommending an RTL implementation strategy, the Shadow understands:

- project objectives,
- architectural constraints,
- implementation history,
- current engineering stage.

Principle 3: Engineer Intent is Authoritative

Statement

The engineer remains the authoritative source of engineering intent.

The Shadow may infer intent when confidence is sufficiently high but shall request clarification whenever uncertainty could significantly affect project understanding.

Rationale

Engineering decisions frequently involve trade-offs that cannot always be inferred reliably from observed activities alone.

The engineer's explanation takes precedence over automated inference.

Example

The Shadow detects a substantial architectural modification.

Rather than assuming the reason, it requests a brief explanation before updating project understanding.

Principle 4: Preserve Engineering Reasoning

Statement

Engineering reasoning shall be treated as a first-class project artifact.

Rationale

Design decisions derive their value from the reasoning that produced them.

Without preserved reasoning, future engineers must reconstruct context through repeated discussions or investigation.

Example

When an architecture changes, the Shadow records not only the new architecture but also the motivation, alternatives considered, and engineering constraints that influenced the decision.

Principle 5: Observe Continuously, Interrupt Selectively

Statement

Observation shall be continuous.

Interruption shall be selective.

Rationale

Most engineering activities provide useful context but do not require immediate interaction.

Interruptions shall occur only when clarification is necessary to preserve accurate project understanding.

Example

Reading research papers requires no interruption.

Replacing a core architectural component may require clarification.

Principle 6: Maintain an Evolving Project Understanding

Statement

The Shadow shall continuously update its understanding as the project evolves.

Project understanding is never considered complete.

Rationale

Engineering projects evolve through experimentation, validation, redesign, and changing priorities.

The Shadow must evolve alongside the project rather than relying on static documentation.

Example

Changes to project objectives, architecture, constraints, verification results, or guide feed-back update the Shadow's understanding.

Principle 7: Communicate with Transparency**Statement**

The Shadow shall clearly distinguish between observed facts, inferred conclusions, and engineering recommendations.

Rationale

Engineers must understand the origin of information before making design decisions.

Transparency increases trust and allows incorrect assumptions to be corrected quickly.

Example

The Shadow communicates whether a conclusion originated from:

- direct engineer input,
- observed engineering activities,
- project artifacts,
- automated inference.

6.4 Behavioral Summary

Engineering Shadow shall behave as a collaborative engineering partner that continuously develops project understanding while minimizing interruption to the engineer's workflow.

Its primary responsibility is to maintain sufficient project understanding to provide context-aware engineering assistance throughout the RTL engineering lifecycle.

The Shadow shall prioritize understanding over automation, reasoning over activity logging, and engineering context over isolated interactions.

7 Product Hypothesis

State the core hypothesis this product aims to validate.

8 MVP

Define only the minimum functionality required for validation.

9 Open Questions

Maintain unanswered product and technical questions here.